YAMLess Case Study — REMS Ingest Manager

YAMLess: Generating Azure Infrastructure from Code

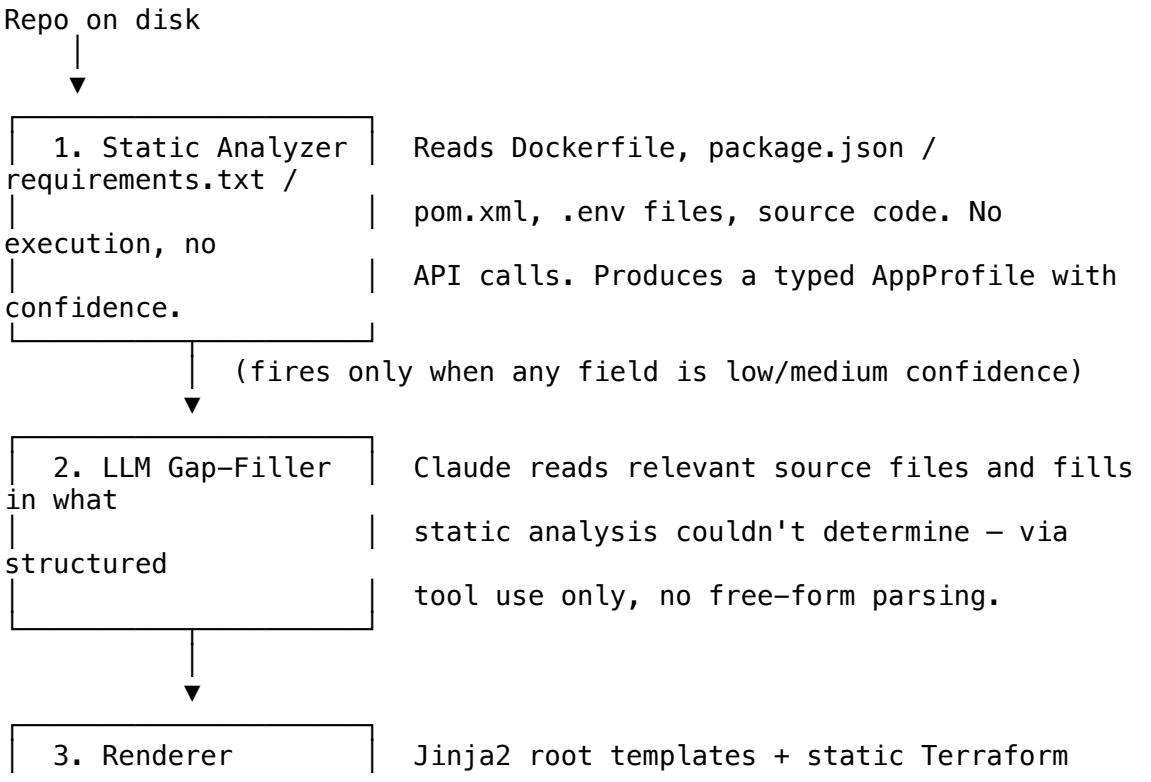
Case Study — krogertechnology/pos-service-instore-rems-ingest-manager

---

What is YAMLess?

YAMLess is an **Infrastructure from Code (IFC)** tool. Given a code repository, it produces a complete, policy-validated Azure Terraform workspace — with no configuration files, no schema to fill in, and no IaC expertise required from the developer.

The pipeline has four stages:



```
modules →
│                      │  ready-to-apply workspace. Outputs Container
Apps
│                      │  or AKS, selected by --target.
└──────────┬───────────┘
           ▼
┌──────────┴───────────┐
│  4. Validator        │  checkov policy-as-code. Security posture is
enforced
│                      │  by static module files — the LLM cannot
affect it.
└──────────────────────┘
```

Applying YAMLess to pos-service-instore-rems-ingest-manager

pos-service-instore-rems-ingest-manager is a Kroger NGPOS in-store accounting service: a **Java 17 / Spring Boot + Apache Camel** background worker that ingests REMS (Retail Expense Management System) data from Kafka topics and publishes transformed events to Azure Service Bus and AMQP queues. It has no HTTP server, no database, and no exposed port — a purely event-driven, headless consumer.

Command run

```
yamless generate /tmp/pos-service-instore-rems-ingest-manager \
  --no-llm \
  --target aks \
  --out-dir ~/yamless-outputs/rems-ingest-manager-infra \
  --validate
```

Profile detected

Field	Detected value	Confidence	Notes
Runtime	`java`	high	FROM `java-openjre:17-latest` in Dockerfile
Runtime version	`17`	high	Parsed from image tag
Port	`null`	—	No EXPOSE directive — correct for a headless consumer
Shape	`background-worker`	high	`camel-kafka`, `camel-azure-`

Field	Detected value	Confidence	Notes
			`servicebus`, `amqp-client` in pom.xml
Database	`null`	—	No DB artifact IDs found — correct
Storage	`false`	—	No storage artifact IDs found — correct
Queue	`true`	high	Azure Service Bus + Kafka + AMQP all detected
Cache	`false`	—	Not detected
Scale trigger	`queue`	—	Derived from queue detection → KEDA ScaledObject
Ingress	`external: false`	—	Correct — no public endpoint
Secrets	`[]`	—	No `.env` files committed; secrets injected at runtime

Output generated (AKS target)

```
rems-ingest-manager-infra/
├── terraform/
│   ├── main.tf              # VNet, AKS cluster, Workload Identity,
Key Vault, Service Bus
│   ├── variables.tf
│   ├── outputs.tf
│   ├── providers.tf
│   └── modules/
│       ├── vnet/            # VNet + AKS subnet (10.0.0.0/16 /
10.0.0.0/20)
│       ├── aks_cluster/     # AKS with OIDC, Workload Identity, Key
Vault CSI, KEDA
│       ├── aks_workload_identity/  # Federated identity credential +
KV Secrets User role
│       ├── keyvault/        # Key Vault with RBAC + network deny-all
│       └── servicebus/      # Service Bus namespace + queue +
connection string → KV
└── k8s/
```

```
    ├── namespace.yaml
    ├── service-account.yaml         # Workload Identity annotation
    ├── secret-provider-class.yaml   # CSI driver mounts
    ├── deployment.yaml              # No HTTP probes — correct for a
worker
    ├── hpa.yaml                     # KEDA ScaledObject with azure-
servicebus trigger
    └── network-policy.yaml          # Default-deny; no ingress allow
rule (worker shape)
```

Note: `service.yaml` and `ingress.yaml` are **not generated** — correct, because the shape is `background-worker`.

43/43 policy checks passed — the output is valid and deployable.

What the output correctly captured

- Headless worker shape: no Service or Ingress Kubernetes manifests
- KEDA `ScaledObject` with `azure-servicebus` trigger (queue-based autoscaling, `min: 1`, `max: 10`)
- Azure Service Bus provisioned with connection string written to Key Vault
- AKS Workload Identity with federated credential for pod-level Key Vault access
- Key Vault CSI driver configured in the AKS cluster
- Runtime tag `java` on all Azure resources for observability
- All security golden standards: RBAC-only KV auth, TLS 1.2, public network access disabled, consistent tagging

What was missed

Missing	Root cause
Kafka / Event Hubs module	`camel-kafka` was detected and used for shape inference, but no Kafka Terraform module exists — only Service Bus. The app consumes from Kafka but the generated infra doesn't provision a Kafka-compatible endpoint.
All runtime secrets	The repo has zero `.env` files. Connection strings (Kafka bootstrap, Service Bus SAS, AMQP credentials, schema registry URL) are injected via Kubernetes Secrets or Helm values at deploy time — neither of which the env scanner currently reads.
Kafka-specific Service Bus distinction	The app uses both Kafka (ingest) and Service Bus (publish). YAMLess detects `queue: true` but cannot yet distinguish which backing service maps to which role.

What this test case drove into YAMLess

Before testing this repo, YAMLess had no Java support. Three bugs were found and fixed:

1. Java runtime was not detected — returned `python` instead

Root cause: The Dockerfile base image is `krogertechnology-docker-prod.jfrog.io/library/java-openjre:17-latest` — a private Kroger JFrog image. YAMLess's `_RUNTIME_PATTERNS` only matched `node:` and `python:` image names, so it fell through to `_infer_runtime_from_files()`.

`_infer_runtime_from_files()` then hit `any(root.glob("**/*.py"))` which matched `pact-broker/normalize-asyncapi.py` — a single utility script with no relation to the application runtime — and classified the entire repo as Python.

Fix applied: - Added Java image patterns to `_RUNTIME_PATTERNS`: `java-openjre`, `openjdk`, `eclipse-temurin`, `corretto`, `temurin`, `liberica`, `zulu`, `adoptopenjdk` — with version capture from the tag - Added `pom.xml` / `build.gradle` / `build.gradle.kts` detection to `_infer_runtime_from_files()`, checked before Python indicators - Changed `root.glob("**/*.py")` → `root.glob("*.py")` (top-level only) — a single script in a subdirectory is not evidence of a Python runtime

2. pom.xml dependency parsing did not exist

Root cause: YAMLess only parsed `package.json` (Node) and `requirements.txt` / `pyproject.toml` (Python). Java repos produced `database: null`, `queue: false`, `shape: unknown` regardless of what was in `pom.xml`.

Fix applied: Added `_parse_java()` to `dependencies.py` using `xml.etree.ElementTree` (stdlib). It extracts all `<artifactId>` values and classifies them against tables for: - Database (`postgresql`, `mysql-connector-j`, `azure-cosmos`, `mssql-jdbc`, …) - Storage (`azure-storage-blob`, `aws-java-sdk-s3`, …) - Queue (`camel-kafka`, `kafka-clients`, `camel-azure-servicebus`, `azure-messaging-servicebus`, `amqp-client`, …) - Cache (`jedis`, `lettuce-core`, `spring-boot-starter-data-redis`, …) - HTTP frameworks (`spring-boot-starter-web`, `spring-webflux`, `quarkus-resteasy`, …) — produces `shape_hint: "http-service"` - Cron schedulers (`quartz`, `camel-quartz`) — produces `shape_hint: "cron-job"` - Queue consumers (Kafka, Service Bus, AMQP) — produces `shape_hint: "background-worker"`

3. Scale trigger was stuck at `cpu` for Java workers

Root cause: The `background-worker` scale trigger selection called `_has_queue_dep(deps_lower)` which checked raw npm package names. For Java, `deps` was always `set()` so the trigger defaulted to `cpu` even when `queue: true` was set.

Fix applied: Added `has_queue: bool = False` parameter to `shape_detector.detect()`. Runner passes the queue signal from dep analysis directly, so `ScaleTrigger.QUEUE` is selected correctly for any runtime with a queue dependency.

Learnings for YAMLess

1. Kafka needs its own Terraform module (backlog #10)

What happened: The analyzer correctly identifies `camel-kafka` and `kafka-streams-avro-serde` as Kafka consumers. But the renderer has only a Service Bus module. There is no Azure Event Hubs module, which is the natural Kafka-compatible endpoint on Azure.

The generated output provisions Service Bus (also used by this service for publishing), but nothing for the Kafka ingest side. A team deploying this output would need to manually provision an Event Hubs namespace with Kafka protocol enabled.

Proposed fix: Add an `event_hubs/` Terraform module to both renderers. Detect Kafka artifact IDs as a separate signal (`has_kafka: bool`) distinct from Service Bus (`has_servicebus: bool`). Wire both conditionally into `main.tf.j2`. The two are not mutually exclusive — this service uses both.

---

2. Runtime-injected secrets are invisible to the env scanner (backlog #11)

What happened: The repo has no `.env` files. All connection strings — Kafka bootstrap server, Service Bus SAS key, AMQP credentials, Schema Registry URL — arrive via Kubernetes Secrets or CI/CD variable injection. The env scanner returned `secrets: []`, which means Key Vault is provisioned but empty. A team deploying this output would need to manually populate every secret.

Proposed fix: Extend secret scanning to cover: - **Helm values** — scan `helm/`, `charts/` for `secretKeyRef`, `configMapKeyRef`, and `env:` key names in `values.yaml` / `values-*.yaml` - **Kubernetes manifests** — scan `k8s/`, `kubernetes/`, `deploy/` for `Secret` and `ConfigMap` env var keys - **Spring Boot `application.yml`** — scan `src/main/resources/application*.yml` for `${ENV_VAR_NAME}` placeholder patterns; this is the standard way Spring Boot apps declare their config surface

For this repo specifically, `src/main/resources/application*.yml` would likely surface the Kafka bootstrap, schema registry, and Service Bus connection string env var names.

Improvement Roadmap

Priority	Change	Impact
Medium	Kafka / Azure Event Hubs Terraform module (#10)	Correct infra for any Kafka-consuming service
Medium	Secret scanning for Helm values + Spring Boot application.yml (#11)	Surfaces runtime-injected secrets for Java and Helm-deployed services

Priority	Change	Impact
Medium	Multi-queue model — distinguish `has_kafka` from `has_servicebus`	One service can use both; they map to different Azure resources

Summary

This is the first Java repo tested against YAMLess. The static analyzer correctly identified: - Runtime: Java 17 (from a private-registry base image containing “java” and “17” in the tag) - Shape: background-worker (from `camel-kafka`, `camel-azure-servicebus`, `amqp-client` in `pom.xml`) - Queue: true (Azure Service Bus + Kafka + AMQP) - Scale trigger: queue (KEDA)

The generated AKS Terraform and Kubernetes manifests are structurally correct — no service, no ingress, KEDA-based autoscaling, Workload Identity, Key Vault. **43/43 checkov checks pass.**

The two gaps are well-bounded: Kafka has no Terraform module yet (tracked as #10), and runtime-injected secrets are not discoverable from committed files (tracked as #11). Both are solvable; neither is a fundamental limitation of the approach.

Testing this repo also uncovered and fixed three bugs in the analyzer: a false Python classification from a `**/*.py` glob, no Java runtime patterns in the Dockerfile detector, and no pom.xml parser. Java support is now part of the core analyzer.

Generated by YAMLess — https://github.com/zafar-ahmed-kr/YAMLess